

MACHINE LEARNING

AIMLCZG565 | BITS Pilani WILPD | S2 2025-26

Complete Study Notes — Sessions 1 to 6

With Worked Examples · Exam Questions · Deep Theory

Mid-Semester: 21 June 2026 (Closed Book) | Comprehensive: 6 Sep 2026 (Open Book)



Document Guide

Each session follows the same structure:

- THEORY → key concepts explained in depth with intuition
- FORMULAS → every important formula with derivation hints
- EXAMPLES → step-by-step worked problems (exam-style numericals)
- EXAM Qs → typical exam questions with model answers
- TIPS → what to write, common mistakes, marks strategy

Sessions Covered: 1-Introduction 2-Math & Workflow 3-Linear Regression 4-Linear Classification 5-Logistic Regression 6-Decision Trees

SESSION 1 Introduction to Machine Learning

1.1 What is Machine Learning?

Machine Learning is a sub-field of Artificial Intelligence where systems learn patterns from data to make predictions or decisions without being explicitly programmed for each task. The key idea: instead of writing rules by hand, we let the machine discover rules from examples.



Tom Mitchell's Formal Definition (1997) — Must Memorise

A computer program is said to LEARN from experience E with respect to some class of Tasks T and Performance measure P if its performance at tasks in T , as measured by P , improves with experience E .

The three components:

- T (Task) — what the program does (classify, predict, cluster, ...)
- P (Performance) — how we measure success (accuracy, RMSE, F1, ...)

E (Experience) — the data the program learns from (labelled or unlabelled)



Example 1.1 — Applying Mitchell's Definition to Spam Filter

Spam Email Classifier:

- T = Classify incoming emails as SPAM or NOT SPAM
- P = Fraction of emails correctly classified (Accuracy)
- E = Large dataset of emails labelled as spam/not-spam

Chess-Playing Program:

- T = Play games of chess
- P = Percentage of games won against opponents
- E = Games played against itself or human opponents

Medical Diagnosis System:

- T = Diagnose whether a patient has diabetes
- P = Sensitivity (recall) + Specificity on held-out patient records
- E = Medical records with confirmed diagnoses

1.2 Types of Machine Learning — Detailed Taxonomy

Type	Supervision	Goal	Examples
Supervised	Full labels (X,Y)	Learn $f: X \rightarrow Y$	Regression, Classification, Object Detection
Unsupervised	No labels (X only)	Find hidden structure	Clustering, PCA, Autoencoders
Semi-Supervised	Few labels + many unlabelled	Combine both	SSL, Label Propagation
Reinforcement	Reward signal	Maximise cumulative reward	Game AI, Robotics, RecSys
Self-Supervised	Labels derived from data	Learn representations	BERT, GPT pre-training
Online Learning	Stream of data	Update model incrementally	Fraud detection, Stock prices

1.3 Designing a Learning System — Four Decisions



The Four Design Choices (Mitchell Ch.1)

1. TRAINING EXPERIENCE — what data does the system learn from?

- Direct feedback: e.g., board position labelled with win/loss
- Indirect feedback: e.g., sequence of moves; only final outcome known
- Teacher: supervised (labels provided) vs no teacher: unsupervised
- Distribution: does training data represent deployment distribution?

2. TARGET FUNCTION — what exactly should be learned?

- Evaluation function $V(b)$ for board state b in checkers
- Conditional probability $P(y|x)$ for classification
- A policy π : state $\rightarrow$ action for reinforcement learning

3. REPRESENTATION — how is the target function expressed?

- Linear combination of features: $V(b) = w_0 + w_1 \cdot f_1 + w_2 \cdot f_2 + \dots + w_n \cdot f_n$
- Neural network, decision tree, k-nearest neighbours, etc.

4. LEARNING ALGORITHM — how to search the hypothesis space?

- Gradient descent, ID3 algorithm, backpropagation, genetic algorithms

1.4 Overfitting vs Underfitting — Core Theory

The Generalisation Problem

OVERFITTING: Model too complex $\rightarrow$ memorises training data $\rightarrow$ poor on new data

Signs: Training error $\ll$ Test error (large gap)

Causes: Too many parameters, too little data, training too long

Fixes: More data, regularisation (L1/L2), dropout, cross-validation, pruning

UNDERFITTING: Model too simple $\rightarrow$ can't even capture training patterns

Signs: Training error is also high

Causes: Too few parameters, wrong model family, features not informative

Fixes: More complex model, better features, reduce regularisation, more epochs

CAPACITY: informal notion of how complex a hypothesis class is

Low capacity $\rightarrow$ underfit; High capacity $\rightarrow$ overfit

Right capacity $\rightarrow$ generalise well (sweet spot)



Example 1.2 — Overfitting vs Underfitting Illustration

Data: points generated by $y = \sin(x) + \text{noise}$

Degree-1 polynomial ($y = w_0 + w_1 \cdot x$):

$\rightarrow$ Straight line that can't capture the curve $\rightarrow$ UNDERFITTING

$\rightarrow$ High bias, low variance

Degree-3 polynomial ($y = w_0 + w_1x + w_2x^2 + w_3x^3$):

$\rightarrow$ Fits sine curve well $\rightarrow$ GOOD FIT (generalises)

$\rightarrow$ Balanced bias and variance

Degree-15 polynomial:

$\rightarrow$ Passes through every training point exactly $\rightarrow$ OVERFITTING

$\rightarrow$ Training MSE ≈ 0 but test MSE is very large

$\rightarrow$ Low bias, high variance

? EXAM Q: Define Machine Learning using Mitchell's definition. Give TWO different examples with T, P, E clearly identified. (6 marks)

Definition: A computer program is said to LEARN from experience E with respect to some class of Tasks T and Performance measure P if its performance at tasks T, as measured by P, improves with experience E.

Example 1 — Handwritten Digit Recognition:

T = Classify handwritten digit images into digits 0–9
P = Classification accuracy on test images
E = MNIST dataset of 60,000 labelled digit images

Example 2 — House Price Prediction:

T = Predict the sale price of a house given its features
P = Root Mean Squared Error (RMSE) on held-out test houses
E = Historical dataset of house features and actual sale prices

SESSION 2 Mathematical Preliminaries & ML Workflow

2.1 Probability & Bayes' Theorem — Deep Dive

Probability Axioms and Rules

Three axioms of probability:

1. $P(A) \geq 0$ for all events A
2. $P(\Omega) = 1$ (sample space has probability 1)
3. Mutually exclusive: $P(A \cup B) = P(A) + P(B)$

Key Rules:

Product rule: $P(A, B) = P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$
Sum rule (marginal): $P(A) = \sum_B P(A, B) = \sum_B P(A|B) \cdot P(B)$
Conditional: $P(A|B) = P(A, B) / P(B)$
Independence: $P(A, B) = P(A) \cdot P(B)$ iff $A \perp B$

Bayes' Theorem

$$P(H|D) = P(D|H) \cdot P(H) / P(D)$$

$P(H)$ = Prior, $P(D|H)$ = Likelihood, $P(H|D)$ = Posterior, $P(D)$ = Evidence (normalising constant)



Example 2.1 — Bayes' Theorem: Medical Diagnosis (Classic Exam Problem)

Problem:

Disease prevalence: $P(\text{Disease}) = 0.01$ (1% of population)
Test sensitivity: $P(\text{Positive} | \text{Disease}) = 0.95$

Test specificity: $P(\text{Negative} \mid \text{No Disease}) = 0.90$
→ $P(\text{Positive} \mid \text{No Disease}) = 1 - 0.90 = 0.10$ (false positive rate)

Question: If test is positive, what is $P(\text{Disease} \mid \text{Positive})$?

Step 1: $P(\text{No Disease}) = 1 - 0.01 = 0.99$

Step 2: $P(\text{Positive}) = P(\text{Pos} \mid \text{Disease}) \cdot P(\text{Disease}) + P(\text{Pos} \mid \text{No Disease}) \cdot P(\text{No Disease})$
 $= 0.95 \times 0.01 + 0.10 \times 0.99$
 $= 0.0095 + 0.099 = 0.1085$

Step 3: Apply Bayes:

$$\begin{aligned} P(\text{Disease} \mid \text{Positive}) &= P(\text{Positive} \mid \text{Disease}) \times P(\text{Disease}) / P(\text{Positive}) \\ &= (0.95 \times 0.01) / 0.1085 \\ &= 0.0095 / 0.1085 \\ &\approx 0.0876 \quad (\text{only } 8.76\%) \end{aligned}$$

Key Insight: Even with a 95% sensitive test, a positive result only means ~9%

chance of disease because the disease is rare (low prior). This is BASE RATE FALLACY.

2.2 Information Theory — Entropy Deep Dive

Shannon Entropy

$$H(X) = -\sum_i P(x_i) \log_2 P(x_i)$$

Units: bits (log base 2) or nats (natural log). Convention: $0 \cdot \log(0) = 0$

Properties of Entropy — Important for Exam

1. $H(X) \geq 0$ always (entropy is non-negative)
2. $H(X) = 0$ iff X is deterministic (one outcome has probability 1)
3. $H(X)$ is MAXIMUM when X is uniformly distributed over K outcomes: $H_{\text{max}} = \log_2(K)$
4. For binary variable: H is max = 1 bit when $P(0)=P(1)=0.5$
5. Entropy measures UNCERTAINTY / SURPRISE / INFORMATION CONTENT
High entropy = high uncertainty = more information gained from observing X

Joint Entropy: $H(X,Y) = -\sum_x \sum_y P(x,y) \log P(x,y)$

Conditional Entropy: $H(Y|X) = \sum_x P(x) H(Y|X=x)$

Chain Rule: $H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$

Mutual Information: $I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$
 $= H(X) + H(Y) - H(X,Y)$

Example 2.2 — Entropy Calculation (Exam Numerical)

Fair coin: $P(H)=0.5$, $P(T)=0.5$

$$H = -0.5 \cdot \log_2(0.5) - 0.5 \cdot \log_2(0.5)$$

$H = -0.5 \cdot (-1) - 0.5 \cdot (-1) = 0.5 + 0.5 = 1.0 \text{ bit}$ ✓ (maximum for binary)

Biased coin: $P(H)=0.9$, $P(T)=0.1$

$$H = -0.9 \cdot \log_2(0.9) - 0.1 \cdot \log_2(0.1)$$

$$H = -0.9 \cdot (-0.152) - 0.1 \cdot (-3.322)$$

$$H = 0.137 + 0.332 = 0.469 \text{ bits} \quad (\text{less uncertain than fair coin})$$

Three-sided die: $P(1)=P(2)=P(3)=1/3$

$$H = -3 \times (1/3) \cdot \log_2(1/3) = -\log_2(1/3) = \log_2(3) \approx 1.585 \text{ bits}$$

KEY TRICK: $\log_2(1/2)=-1$, $\log_2(1/4)=-2$, $\log_2(1/8)=-3$, $\log_2(1/3) \approx -1.585$

2.3 Performance Metrics — Worked Examples

Example 2.3 — Confusion Matrix Analysis

A classifier on 100 test examples gives:

True Positives (TP) = 40 (correctly said positive, actually positive)

True Negatives (TN) = 30 (correctly said negative, actually negative)

False Positives (FP) = 10 (said positive, actually negative) ← Type I Error

False Negatives (FN) = 20 (said negative, actually positive) ← Type II Error

Calculations:

$$\text{Accuracy} = (TP+TN)/(TP+TN+FP+FN) = (40+30)/100 = 70/100 = 0.70$$

$$\text{Precision} = TP/(TP+FP) = 40/(40+10) = 40/50 = 0.80$$

$$\text{Recall} = TP/(TP+FN) = 40/(40+20) = 40/60 = 0.667$$

$$\begin{aligned} \text{F1 Score} &= 2 \times P \times R / (P+R) = 2 \times 0.80 \times 0.667 / (0.80+0.667) \\ &= 1.067/1.467 = 0.727 \end{aligned}$$

$$\text{Specificity} = TN/(TN+FP) = 30/40 = 0.75$$

When to use which metric?

Accuracy: balanced classes, overall correctness matters

Precision: FP is costly (spam detection — don't want to block real emails)

Recall: FN is costly (cancer detection — don't want to miss a case)

F1: when both FP and FN costs matter equally

? EXAM Q: A cancer screening test gives TP=45, FP=5, FN=15, TN=35. Compute Accuracy, Precision, Recall, F1. Which metric is most important here and why? (8 marks)

Total = 45+5+15+35 = 100

Accuracy = (45+35)/100 = 80%

Precision = 45/(45+5) = 45/50 = 90%

Recall = 45/(45+15) = 45/60 = 75%

F1 Score = $2 \times 0.90 \times 0.75 / (0.90 + 0.75) = 1.35 / 1.65 = 81.8\%$

Most important: RECALL (Sensitivity). In cancer screening, a False Negative (missing an actual cancer patient) is far more dangerous than a False Positive (unnecessary follow-up test). We want to catch every cancer case, even at cost of some false alarms. Here FN=15 means 15 cancer patients were missed — that is unacceptable in practice. We should tune the threshold to increase Recall, accepting lower Precision.

SESSION 3 Linear Models for Regression

3.1 Linear Regression — Deep Theory

Why Minimise Squared Error? (Maximum Likelihood Justification)

Assume: $y = w^T x + \varepsilon$ where $\varepsilon \sim N(0, \sigma^2)$ (Gaussian noise)

Then: $P(y|x, w) = N(y; w^T x, \sigma^2) = (1/\sqrt{2\pi\sigma^2}) \exp(-(y - w^T x)^2 / 2\sigma^2)$

Maximum Likelihood Estimation (MLE): maximise $P(D|w) = \prod_i P(y_i|x_i, w)$

Taking log and maximising:

$$\log P(D|w) = -n/2 \log(2\pi\sigma^2) - (1/2\sigma^2) \sum_i (y_i - w^T x_i)^2$$

Maximising log-likelihood $\equiv$ MINIMISING $\sum_i (y_i - w^T x_i)^2 \equiv$ Minimising MSE

CONCLUSION: MSE is not arbitrary! It is the MLE under Gaussian noise assumption. If noise is Laplacian, MAE (L1 loss) is the MLE — this gives LASSO regression.

3.2 Normal Equations — Derivation + Example

Normal Equation

$$w^* = (X^T X)^{-1} X^T y$$

Exact analytical solution. One-step computation. $O(d^3)$ complexity.

Derivation of Normal Equation

Loss: $J(w) = (1/2) \|y - Xw\|^2 = (1/2)(y - Xw)^T (y - Xw)$

Expand: $= (1/2)[y^T y - y^T X w - w^T X^T y + w^T X^T X w]$
 $= (1/2)[y^T y - 2w^T X^T y + w^T X^T X w]$ (since $y^T X w$ is scalar $= w^T X^T y$)

Gradient: $\partial J / \partial w = -X^T y + X^T X w$

Set gradient to zero: $X^T X w = X^T y \leftarrow$ these are the NORMAL EQUATIONS

Solve: $w^* = (X^T X)^{-1} X^T y$ (if $X^T X$ is invertible)

Note: $X^T X$ is $d \times d$ matrix. Inversion costs $O(d^3)$ — expensive if d is large.
Ridge regression adds λI to make it always invertible: $w = (X^T X + \lambda I)^{-1} X^T y$



Example 3.1 — Normal Equation: Full Numerical Solution

Data: 3 training points

x: [1, 2, 3] (feature, 1D for simplicity)
y: [2, 4, 5] (target)

Design matrix X (add column of 1s for bias):

X = $\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$, y = $\begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix}$

Step 1: Compute $X^T X$

$X^T X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \times \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$
 $= \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$

Step 2: Compute $X^T y$

$X^T y = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \times [2, 4, 5]^T = [11, 23]^T$

Step 3: Solve $(X^T X)w = X^T y$

$\begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} \times [w_0, w_1]^T = [11, 23]^T$

$3w_0 + 6w_1 = 11 \quad \dots (i)$

$6w_0 + 14w_1 = 23 \quad \dots (ii)$

From (i): $w_0 = (11 - 6w_1)/3$

Sub in (ii): $6(11 - 6w_1)/3 + 14w_1 = 23 \rightarrow 22 - 12w_1 + 14w_1 = 23 \rightarrow 2w_1 = 1 \rightarrow w_1 = 0.5$

$w_0 = (11 - 6 \times 0.5)/3 = (11 - 3)/3 = 8/3 \approx 0.667$

Result: $\hat{y} = 0.667 + 0.5 \cdot x$

Predictions: $\hat{y}_1=1.167, \hat{y}_2=1.667, \hat{y}_3=2.167$ MSE = check against $y=[2, 4, 5]$

3.3 Gradient Descent — Complete Walkthrough

Batch Gradient Descent

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot (1/n) \cdot \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

α = learning rate. Repeat until $\|\nabla J\| < \epsilon$ or max epochs reached.

Example 3.2 — Two Iterations of Gradient Descent (Exam Style)

Data: $\mathbf{x} = [1, 2]$, $\mathbf{y} = [3, 5]$. Learning rate $\alpha = 0.1$

Design matrix (with bias $x_0=1$):

$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$, $\mathbf{y} = [3, 5]$, init $\mathbf{w} = [0, 0]^T$

=== ITERATION 1 ===

Step 1: Predictions $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \cdot [0, 0]^T = [0, 0]$

Step 2: Residuals $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y} = [0-3, 0-5] = [-3, -5]$

Step 3: Gradient $\nabla J = (1/2) \cdot \mathbf{X}^T \cdot \mathbf{e}$
 $= (1/2) \cdot \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}^T \cdot [-3, -5]$
 $= (1/2) \cdot [(-3)+(-5), (-3)+(-10)]$
 $= (1/2) \cdot [-8, -13] = [-4, -6.5]$

Step 4: Update $\mathbf{w} \leftarrow [0, 0] - 0.1 \cdot [-4, -6.5] = [0.4, 0.65]$

=== ITERATION 2 ===

Step 1: Predictions $\hat{\mathbf{y}} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \cdot [0.4, 0.65]^T = [1.05, 1.70]$

Step 2: Residuals $\mathbf{e} = [1.05-3, 1.70-5] = [-1.95, -3.30]$

Step 3: Gradient $\nabla J = (1/2) \cdot [(-1.95)+(-3.30), (-1.95)+(-6.60)]$
 $= (1/2) \cdot [-5.25, -8.55] = [-2.625, -4.275]$

Step 4: Update $\mathbf{w} \leftarrow [0.4, 0.65] - 0.1 \cdot [-2.625, -4.275]$
 $= [0.4+0.2625, 0.65+0.4275] = [0.6625, 1.0775]$

Convergence target: true $w_0=1$, $w_1=2$ (since $y=1+2x$). GD approaching it!

3.4 Bias-Variance Tradeoff — Theory + Example

Formal Bias-Variance Decomposition

For any point x_0 , the expected test error of model $\hat{y}(x_0)$ is:

$$\begin{aligned} E_D[(y - \hat{y})^2] &= [E_D[\hat{y}] - f(x_0)]^2 + E_D[(\hat{y} - E_D[\hat{y}])^2] + \sigma^2 \\ &= \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise} \end{aligned}$$

Where:

y = true output (with noise) = $f(x_0) + \epsilon$, $\epsilon \sim N(0, \sigma^2)$

$f(x_0)$ = true underlying function (unknown)

$\hat{y}$ = model's prediction (depends on training set D)

E_D = expectation over all possible training sets

Bias²: How far is average prediction from true function?

Variance: How much does prediction vary across different training sets?

Noise (σ^2): Irreducible — can't be removed even with perfect model

The dilemma: reducing bias usually increases variance and vice versa.
Goal: find model complexity that minimises $\text{Bias}^2 + \text{Variance}$.

Example 3.3 — Bias-Variance with Polynomial Regression

True function: $f(x) = \sin(\pi x)$, $x \in [0, 1]$

Noise: $\varepsilon \sim N(0, 0.1^2)$

Experiment: train 100 different datasets, measure bias^2 and variance

Degree-1 polynomial (2 parameters):

$\text{Bias}^2 \approx 0.21$ (high — can't fit a curve with a line)

Variance ≈ 0.003 (low — line doesn't change much with different data)

Total $\approx 0.213 + 0.01(\text{noise})$

Degree-3 polynomial (4 parameters):

$\text{Bias}^2 \approx 0.002$ (low — cubic approximates sine well)

Variance ≈ 0.008 (moderate)

Total $\approx 0.010 + 0.01 \leftarrow \text{MINIMUM TOTAL ERROR (good model)}$

Degree-15 polynomial (16 parameters):

$\text{Bias}^2 \approx 0.0005$ (very low)

Variance ≈ 0.25 (VERY HIGH — wiggles around data)

Total $\approx 0.251 + 0.01 \leftarrow \text{WORST (overfitting)}$

Lesson: Degree-3 is the sweet spot for this problem.

? EXAM Q: Explain the Bias-Variance tradeoff. A model has training error=2% and test error=18%. What does this indicate? How would you fix it? (6 marks)

The expected prediction error decomposes as: $E[\text{error}] = \text{Bias}^2 + \text{Variance} + \text{Noise}$

Bias: systematic error from wrong assumptions — model consistently wrong

Variance: sensitivity to training data — model learns noise, not signal

Training error=2%, Test error=18% $\rightarrow$ Gap = 16%

This large gap indicates HIGH VARIANCE (OVERFITTING)

The model has memorised the training data and fails to generalise

Fixes:

1. Collect more training data — most effective fix
2. Add regularisation: L2 (Ridge) or L1 (Lasso) penalise large weights
3. Reduce model complexity: fewer parameters, shallower tree
4. Feature selection: remove irrelevant features
5. Use cross-validation: detect overfitting during training
6. Early stopping: stop training when validation error stops decreasing

SESSION 4 Linear Models for Classification

4.1 Decision Theory — Deep Theory

Bayesian Decision Theory Framework

We want to make decisions that minimise expected loss (or maximise expected utility).

Setup:

- K classes: $C_1, C_2, \dots, C_k$
- Feature vector: $x \in \mathbb{R}^d$
- Loss matrix: $L(C_j, C_k)$ = cost of predicting C_k when true class is C_j

Expected loss (risk) for assigning x to class C_k :

$$R(C_k|x) = \sum_j L(C_j, C_k) \cdot P(C_j|x)$$

Optimal Bayes Decision: assign to class that MINIMISES expected risk

$$\hat{y} = \operatorname{argmin}_k R(C_k|x)$$

For 0-1 loss (equal cost for all errors):

$$L(C_j, C_k) = 0 \text{ if } j=k, \text{ else } 1$$

Expected risk = $1 - P(C_k|x)$ (probability of being wrong)

Minimising risk = MAXIMISING $P(C_k|x)$ $\leftarrow$ MAP/Bayes rule

This is why the Bayes Optimal Classifier picks the class with highest posterior.

4.2 Discriminant Functions — Geometry

Linear Discriminant: Geometric Interpretation

For binary classification with $g(x) = w^T x + w_0$:

Decision boundary: $w^T x + w_0 = 0 \leftarrow$ a $(d-1)$ -dimensional hyperplane

Key properties:

- w is the NORMAL VECTOR to the decision boundary (perpendicular to it)
- $\|w\|$ normalises the direction; w_0 (bias) shifts the boundary from origin
- Signed distance from point x to boundary: $r = g(x)/\|w\|$
- Sign of $g(x)$ tells which side of the hyperplane x is on

For K classes, we need K discriminant functions:

$$g_k(x) = w_k^T x + w_{k0} \text{ for } k=1, \dots, K$$

Decision: assign x to class $k^* = \operatorname{argmax}_k g_k(x)$

The decision boundaries between class i and j are where $g_i(x) = g_j(x)$

$$(w_i - w_j)^T x + (w_{i0} - w_{j0}) = 0 \leftarrow \text{also a hyperplane}$$

4.3 Fisher's Linear Discriminant Analysis (LDA)

Fisher Criterion (Maximise)

$$J(\mathbf{w}) = (\mathbf{w}^T \mathbf{S}_B \mathbf{w}) / (\mathbf{w}^T \mathbf{S}_W \mathbf{w})$$

$\mathbf{S}_B$ = between-class scatter, $\mathbf{S}_W$ = within-class scatter

Computing LDA — Step by Step Theory

Given two classes with n_1 and n_2 examples:

1. Compute class means:

$$\mu_1 = (1/n_1) \sum_{x \in C_1} x, \quad \mu_2 = (1/n_2) \sum_{x \in C_2} x$$

2. Within-class scatter matrix (pooled covariance $\times n$):

$$\mathbf{S}_W = \sum_{x \in C_1} (x - \mu_1)(x - \mu_1)^T + \sum_{x \in C_2} (x - \mu_2)(x - \mu_2)^T$$

3. Between-class scatter matrix:

$$\mathbf{S}_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$$

4. Optimal projection: $\mathbf{w}^* = \mathbf{S}_W^{-1}(\mu_1 - \mu_2)$ [Fisher's solution]

This is found by solving generalised eigenvalue problem: $\mathbf{S}_W^{-1} \cdot \mathbf{S}_B \cdot \mathbf{w} = \lambda \mathbf{w}$

5. Project data: $z = \mathbf{w}^T x$ (1D projection)

Threshold: $t = (n_1 \tilde{\mu}_1 + n_2 \tilde{\mu}_2) / (n_1 + n_2)$ where $\tilde{\mu}_k = \mathbf{w}^T \mu_k$

LDA is both a classifier AND a dimensionality reduction technique.

For K classes, can project to at most $K-1$ dimensions.

Example 4.1 — LDA Worked (2D to 1D Projection)

Class 1: points $(1,2), (2,1), (1,1) \rightarrow \mu_1 = (4/3, 4/3)$

Class 2: points $(5,5), (6,4), (5,6) \rightarrow \mu_2 = (16/3, 15/3) = (5.33, 5.0)$

$\mathbf{S}_W$ computation (sketch):

For C_1 : deviations from $\mu_1 = (1.33, 1.33)$:

$(1-1.33, 2-1.33) = (-0.33, 0.67), (2-1.33, 1-1.33) = (0.67, -0.33), (1-1.33, 1-1.33) = (-0.33, -0.33)$

$\mathbf{S}_W$ = sum of outer products (xx for C_1 and C_2)

[Approximate] $\mathbf{S}_W \approx \begin{bmatrix} 0.67 & -0.33 \\ -0.33 & 0.67 \end{bmatrix} + \begin{bmatrix} 0.67 & -0.33 \\ -0.33 & 0.67 \end{bmatrix}$

$\mathbf{S}_W \approx \begin{bmatrix} 1.33 & -0.67 \\ -0.67 & 1.33 \end{bmatrix}$

Direction: $\mu_1 - \mu_2 = (1.33-5.33, 1.33-5.0) = (-4.0, -3.67)$

$\mathbf{w}^* = \mathbf{S}_W^{-1}(\mu_1 - \mu_2) \rightarrow$ proportional to $[-4.0, -3.67]$ after $\mathbf{S}_W^{-1}$ transform

Key: w^* points FROM class 2 TO class 1, so we classify based on sign of $w^T x$.

4.4 Generative vs Discriminative — Comparison Theory

Why Both Approaches Matter

GENERATIVE MODELS model $P(x,y) = P(x|y) \cdot P(y)$:

- + Can generate new data samples (useful for data augmentation)
- + Handle missing features naturally via marginalisation
- + Work with less labelled data (can use unlabelled data too)
- + Can incorporate prior knowledge about $P(x|y)$
- Must model $P(x|y)$ which is hard in high dimensions
- Usually lower discriminative accuracy than discriminative models

DISCRIMINATIVE MODELS model $P(y|x)$ directly:

- + No need to model complex $P(x|y)$
- + Generally achieve higher classification accuracy
- + Can use more flexible features (don't need generative story)
- Cannot generate new data
- Need fully labelled data
- Less interpretable about data structure

Examples:

Generative: Naive Bayes, LDA, GMM, Hidden Markov Models

Discriminative: Logistic Regression, SVM, Neural Networks, CRF

? EXAM Q: Explain the difference between generative and discriminative classifiers. Give one example of each. Which generally gives higher accuracy and why? (6 marks)

Generative classifiers model the joint distribution $P(x,y) = P(x|y) \cdot P(y)$ and use Bayes theorem to compute $P(y|x)$ for prediction.

Example: Naive Bayes classifier — models $P(x|\text{class})$ separately for each class.

Discriminative classifiers directly model $P(y|x)$ or the decision boundary.

Example: Logistic Regression — directly models $P(y=1|x) = \sigma(w^T x)$.

Discriminative models generally give higher classification accuracy because:

1. They don't spend capacity modelling $P(x)$ which is not needed for classification
2. They directly optimise the quantity we care about: $P(y|x)$
3. They are less constrained by generative assumptions (e.g., feature independence)

However, generative models can work better with very little labelled data since they can also learn from unlabelled data via $P(x)$.

SESSION 5 Logistic Regression — In Depth

5.1 Sigmoid Function — Properties

Sigmoid / Logistic Function — All Properties

$$\sigma(z) = 1 / (1 + e^{-z})$$

Key properties (all examinable):

1. Range: $\sigma(z) \in (0, 1)$ — always a valid probability
2. $\sigma(0) = 0.5$ — decision boundary at $z=0$
3. $\sigma(-z) = 1 - \sigma(z)$ — symmetric around $(0, 0.5)$
4. As $z \rightarrow +\infty$: $\sigma \rightarrow 1$; as $z \rightarrow -\infty$: $\sigma \rightarrow 0$
5. Derivative: $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$ ← used in backpropagation!

Why sigmoid for classification?

- Maps any real number to $(0, 1)$ — interpretable as probability
- Smooth and differentiable — gradient descent works
- Squashes extreme values — numerically stable
- Monotone — preserves ordering

The odds ratio and log-odds (logit):

$$\text{If } P(y=1|x) = \sigma(w^T x) = p$$

$$\text{Then: } p/(1-p) = e^{(w^T x)} \leftarrow \text{odds ratio}$$

$$\text{And: } \log(p/(1-p)) = w^T x \leftarrow \text{log-odds is linear in } x$$

This is why it's called LOGISTIC regression — log-odds is linear!

5.2 Log-Loss Derivation — From Scratch

Why Cross-Entropy? MLE Derivation

$$\text{Model: } P(y=1|x, w) = \sigma(w^T x) = \hat{y}$$

For binary $y \in \{0, 1\}$, we use Bernoulli likelihood:

$$P(y|x, w) = \hat{y}^y \cdot (1 - \hat{y})^{(1-y)}$$

For n independent training examples, joint likelihood:

$$L(w) = \prod_i \hat{y}_i^{y_i} \cdot (1 - \hat{y}_i)^{(1-y_i)}$$

Log-likelihood (easier to maximise, same solution):

$$\log L(w) = \sum_i [y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)]$$

Negative log-likelihood (we MINIMISE this = Cross-Entropy Loss):

$$J(w) = -(1/n) \sum_i [y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)]$$

This is called Binary Cross-Entropy (BCE) or Log-Loss.

Minimising $J(w)$ is exactly MAXIMUM LIKELIHOOD ESTIMATION for logistic regression.

Example 5.1 — Computing Log-Loss

Predictions and true labels for 4 examples:

i=1: $y=1, \hat{y}=0.9 \rightarrow$ contribution: $-(1 \cdot \log 0.9 + 0 \cdot \log 0.1) = -\log(0.9) = 0.105$

i=2: $y=1, \hat{y}=0.3 \rightarrow$ contribution: $-(1 \cdot \log 0.3 + 0 \cdot \log 0.7) = -\log(0.3) = 1.204$

i=3: $y=0, \hat{y}=0.1 \rightarrow$ contribution: $-(0 \cdot \log 0.1 + 1 \cdot \log 0.9) = -\log(0.9) = 0.105$

i=4: $y=0, \hat{y}=0.8 \rightarrow$ contribution: $-(0 \cdot \log 0.8 + 1 \cdot \log 0.2) = -\log(0.2) = 1.609$

Total Log-Loss = $(0.105 + 1.204 + 0.105 + 1.609) / 4 = 3.023/4 = 0.756$

Observations:

- i=2 and i=4: confident wrong predictions $\rightarrow$ HIGH loss (1.2, 1.6)
- i=1 and i=3: confident correct predictions $\rightarrow$ LOW loss (0.1, 0.1)
- Cross-entropy penalises confident mistakes much more than MSE would
- As $\hat{y} \rightarrow 0$ when $y=1$: loss = $-\log(0) \rightarrow \infty$ (infinite penalty)

5.3 Full Logistic Regression — Worked Exam Example

Example 5.2 — Logistic Regression: Train + Predict

Training data (2 features x_1, x_2):

Sample 1: $x=(1,1), y=1$

Sample 2: $x=(2,3), y=1$

Sample 3: $x=(3,1), y=0$

Sample 4: $x=(4,2), y=0$

Initial weights: $w=[w_0, w_1, w_2]=[0, 0, 0], \alpha=0.1$

=== ONE GRADIENT DESCENT STEP ===

Compute $z = w_0 + w_1x_1 + w_2x_2$ for each sample ($z=0$ initially for all):

$\sigma(0) = 0.5$ for all samples

$\hat{y} = [0.5, 0.5, 0.5, 0.5]$

Errors ($\hat{y} - y$):

$e_1=0.5-1=-0.5, e_2=0.5-1=-0.5, e_3=0.5-0=0.5, e_4=0.5-0=0.5$

Gradient for each weight:

$\partial J / \partial w_0 = (1/4) (e_1 + e_2 + e_3 + e_4) = (1/4) (-0.5 - 0.5 + 0.5 + 0.5) = 0$

$\partial J / \partial w_1 = (1/4) (e_1 \cdot 1 + e_2 \cdot 2 + e_3 \cdot 3 + e_4 \cdot 4)$
 $= (1/4) (-0.5 \cdot 1 - 0.5 \cdot 2 + 0.5 \cdot 3 + 0.5 \cdot 4) = (1/4) (-0.5 - 1 + 1.5 + 2) = (1/4) (2) = 0.5$

$\partial J / \partial w_2 = (1/4) (e_1 \cdot 1 + e_2 \cdot 3 + e_3 \cdot 1 + e_4 \cdot 2)$

$$= (1/4) (-0.5 \cdot 1 - 0.5 \cdot 3 + 0.5 \cdot 1 + 0.5 \cdot 2) = (1/4) (-0.5 - 1.5 + 0.5 + 1) = (1/4) (-1) = -0.25$$

Weight update:

$$w_0 \leftarrow 0 - 0.1 \times 0 = 0$$

$w_1 \leftarrow 0 - 0.1 \times 0.5 = -0.05$ (higher $x_1 \rightarrow$ lower probability $\rightarrow$ makes sense, class 0 has high x_1)

$$w_2 \leftarrow 0 - 0.1 \times (-0.25) = 0.025$$

5.4 Regularisation — Theory and Effect

L1 vs L2 Regularisation — Why They Behave Differently

L2 Regularisation (Ridge):

$$J(w) = \text{BCE}(w) + \lambda \sum_j w_j^2$$

$$\text{Gradient: } \partial J / \partial w_j = \partial \text{BCE} / \partial w_j + 2\lambda w_j$$

- $\rightarrow$ Each weight is shrunk proportionally: $w_j \leftarrow w_j(1 - 2\alpha\lambda)$ per step
- $\rightarrow$ Weights approach but NEVER reach exactly 0 (continuous shrinkage)
- $\rightarrow$ Bayesian interpretation: Gaussian prior $N(0, 1/2\lambda)$ on weights
- $\rightarrow$ Solution always unique, always exists

L1 Regularisation (Lasso):

$$J(w) = \text{BCE}(w) + \lambda \sum_j |w_j|$$

$$\text{Gradient: } \partial J / \partial w_j = \partial \text{BCE} / \partial w_j + \lambda \cdot \text{sign}(w_j)$$

- $\rightarrow$ Applies constant force toward 0 regardless of weight magnitude
- $\rightarrow$ Weights CAN reach exactly 0 $\rightarrow$ automatic feature selection!
- $\rightarrow$ Bayesian interpretation: Laplacian prior on weights
- $\rightarrow$ Solution may not be unique; computationally harder

Intuition: L2 is like a spring (restoring force $\propto$ displacement).

L1 is like friction (constant force regardless of position).

Friction $\rightarrow$ you stop completely; spring $\rightarrow$ you slow down but never stop.

Example 5.3 — Effect of Regularisation on Weights

Unregularised logistic regression on high-dimensional data (1000 features):

Many weights become large $\rightarrow$ overfitting

Test accuracy = 72%

With L2 regularisation ($\lambda=0.01$):

All 1000 weights shrink toward 0, none reach exactly 0

Test accuracy = 85% (reduced overfitting)

With L1 regularisation ($\lambda=0.01$):

950 of 1000 weights = exactly 0 (sparse solution!)

Only 50 features are used (automatic feature selection)

Test accuracy = 83% (also reduced overfitting + interpretable model)

Choosing λ : use cross-validation

Small λ (0.001): mild regularisation, may still overfit

Large λ (10.0): strong regularisation, may underfit

Optimal λ found by validation curve (error vs λ plot)

? EXAM Q: Derive the gradient of log-loss with respect to weights w for logistic regression. Show it has the same form as linear regression. (8 marks)

Log-loss: $J(w) = -(1/n) \sum_i [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$ where $\hat{y}_i = \sigma(w^T x_i)$

$$\partial J / \partial w = -(1/n) \sum_i [y_i (\partial / \partial w) \log(\hat{y}_i) + (1-y_i) (\partial / \partial w) \log(1-\hat{y}_i)]$$

Using chain rule: $\partial \log(\hat{y}_i) / \partial w = (1/\hat{y}_i) \cdot (\partial \hat{y}_i / \partial w)$

And sigmoid derivative: $\partial \sigma(z) / \partial z = \sigma(z) \cdot (1-\sigma(z)) = \hat{y}_i (1-\hat{y}_i)$

So: $\partial \hat{y}_i / \partial w = \hat{y}_i (1-\hat{y}_i) \cdot x_i$

Substituting:

$$\begin{aligned} \partial J / \partial w &= -(1/n) \sum_i [y_i \cdot (1-\hat{y}_i) \cdot x_i - (1-y_i) \cdot \hat{y}_i \cdot x_i] \\ &= -(1/n) \sum_i [y_i x_i - y_i \hat{y}_i x_i - \hat{y}_i x_i + y_i \hat{y}_i x_i] \\ &= -(1/n) \sum_i [y_i x_i - \hat{y}_i x_i] \\ &= (1/n) \sum_i (\hat{y}_i - y_i) x_i = (1/n) X^T (\hat{y} - y) \end{aligned}$$

This is IDENTICAL in form to linear regression gradient: $(1/n) X^T (Xw - y)$

The only difference is $\hat{y} = \sigma(Xw)$ for logistic vs $\hat{y} = Xw$ for linear.

SESSION 6 Decision Trees

6.1 Decision Tree — Concept and Structure

How Decision Trees Work — Core Theory

A decision tree is a hierarchical structure where:

- Internal nodes: test on a single feature (binary or multi-way split)
- Edges: outcomes of the test
- Leaf nodes: class label (classification) or value (regression)

The algorithm (ID3/C4.5/CART) is GREEDY and TOP-DOWN:

At each step, choose the BEST feature to split on (locally optimal)

Recurse on each child subset

No backtracking — greedy approach

Stopping criteria (any one met → make a leaf):

1. All examples in node have same class (entropy = 0, pure node)

2. No remaining features to split on
3. No examples reach this node
4. Depth limit reached (pre-pruning)
5. Node has too few examples (min_samples_split parameter)

Leaf label assignment:

Classification: majority class label of training examples at leaf

Regression: mean value of training examples at leaf

6.2 ID3 Algorithm — Complete Walkthrough

Information Gain

$$IG(S, A) = H(S) - \sum_v (|S_v|/|S|) \cdot H(S_v)$$

Choose attribute A with maximum IG . This is the ID3 algorithm's core rule.

Example 6.1 — Full ID3 on Play Tennis Dataset

14 examples: 9 Yes (play), 5 No (don't play)

Features: Outlook(S/O/R), Temperature(H/M/C), Humidity(H/N), Wind(W/S)

STEP 1: Compute $H(S)$ for root

$$P(\text{Yes}) = 9/14, P(\text{No}) = 5/14$$

$$\begin{aligned} H(S) &= -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) \\ &= -(0.643)(-0.637) - (0.357)(-1.485) \\ &= 0.409 + 0.530 = 0.940 \text{ bits} \end{aligned}$$

STEP 2: IG for Outlook (Sunny:5, Overcast:4, Rain:5)

$$H(S_{\text{Sunny}}) = -(2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.971 \quad [2\text{Yes}, 3\text{No}]$$

$$H(S_{\text{Overcast}}) = -(4/4)\log_2(4/4) = 0 \quad [4\text{Yes}, 0\text{No}] \text{ PURE!}$$

$$H(S_{\text{Rain}}) = -(3/5)\log_2(3/5) - (2/5)\log_2(2/5) = 0.971 \quad [3\text{Yes}, 2\text{No}]$$

$$\begin{aligned} H(S|\text{Outlook}) &= 5/14 \times 0.971 + 4/14 \times 0 + 5/14 \times 0.971 \\ &= 0.347 + 0 + 0.347 = 0.693 \end{aligned}$$

$$IG(S, \text{Outlook}) = 0.940 - 0.693 = 0.247$$

STEP 3: IG for Humidity (High:7, Normal:7)

$$H(S_{\text{High}}) = -(3/7)\log_2(3/7) - (4/7)\log_2(4/7) = 0.985 \quad [3\text{Yes}, 4\text{No}]$$

$$H(S_{\text{Normal}}) = -(6/7)\log_2(6/7) - (1/7)\log_2(1/7) = 0.592 \quad [6\text{Yes}, 1\text{No}]$$

$$H(S|\text{Humidity}) = 7/14 \times 0.985 + 7/14 \times 0.592 = 0.788$$

$$IG(S, \text{Humidity}) = 0.940 - 0.788 = 0.152$$

STEP 4: IG for Wind (Weak:8, Strong:6)

$$H(S_{\text{Weak}}) = -(6/8)\log_2(6/8) - (2/8)\log_2(2/8) = 0.811 \quad [6\text{Yes}, 2\text{No}]$$

$$H(S_{\text{Strong}}) = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = 1.000 \quad [3\text{Yes}, 3\text{No}]$$

$$H(S|\text{Wind}) = 8/14 \times 0.811 + 6/14 \times 1.000 = 0.463 + 0.429 = 0.892$$

$$IG(S, \text{Wind}) = 0.940 - 0.892 = 0.048$$

SUMMARY OF IG VALUES:

Outlook = 0.247 ← HIGHEST → chosen as ROOT
Humidity = 0.152
Wind = 0.048
Temp = 0.029 (not computed above)

STEP 5: Recurse on Outlook branches

Sunny (5 examples, 2Yes, 3No): recurse with remaining features
Overcast (4 examples, all Yes): LEAF = Yes (pure!)
Rain (5 examples, 3Yes, 2No): recurse with remaining features

Final tree root: Outlook → {Sunny: subtree, Overcast: YES, Rain: subtree}

6.3 Gini Impurity — ID vs CART Comparison



Example 6.2 — Gini Impurity Calculation

Node with 10 examples: 6 Class A, 4 Class B

$$p(A) = 6/10 = 0.6, \quad p(B) = 4/10 = 0.4$$

$$\begin{aligned} \text{Entropy: } H &= -(0.6)\log_2(0.6) - (0.4)\log_2(0.4) \\ &= -(0.6)(-0.737) - (0.4)(-1.322) \\ &= 0.442 + 0.529 = 0.971 \end{aligned}$$

$$\text{Gini: } G = 1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 1 - 0.52 = 0.48$$

After split on feature A:

Left child: 5 examples (5A, 0B) → Gini = $1 - (1^2 + 0^2) = 0$ (PURE)

Right child: 5 examples (1A, 4B) → Gini = $1 - (0.2^2 + 0.8^2) = 1 - (0.04 + 0.64) = 0.32$

$$\text{Weighted Gini after split} = 5/10 \times 0 + 5/10 \times 0.32 = 0.16$$

$$\text{Gini Gain} = 0.48 - 0.16 = 0.32 \rightarrow \text{large gain} \rightarrow \text{good split}$$

COMPARISON ENTROPY vs GINI:

Both measure impurity; Gini is computationally cheaper (no log)
Gini tends to isolate largest class; Entropy more balanced splits
In practice results are similar; sklearn uses Gini by default (CART)

6.4 Overfitting in Decision Trees — Theory



Why Decision Trees Overfit and How to Control It

A fully grown tree can represent ANY function of training data (perfect fit)

→ It memorises noise and mislabelled examples → zero training error

→ But test error is large (high variance)

Two strategies to prevent overfitting:

PRE-PRUNING (Early Stopping):

Stop growing when IG < threshold (e.g., 0.001)

Limit max_depth (e.g., max_depth=5)

Require min_samples_split examples to split (e.g., 20)

Require min_samples_leaf at each leaf (e.g., 5)

PROBLEM: Might stop too early (greedy — a useless split might unlock good one later)

POST-PRUNING (Reduced Error Pruning — Exam Important):

1. Grow full tree
2. Split data: train set (grow tree) + validation set (for pruning)
3. For each internal node (bottom-up order):
 - a. Tentatively replace subtree with leaf (majority class)
 - b. Measure accuracy on validation set
 - c. If accuracy doesn't decrease (or improves) → PRUNE permanently
 - d. Otherwise → keep subtree
4. Repeat until no more pruning improves validation accuracy

Why bottom-up? Pruning lower nodes first avoids pruning parent when child might already be prunable.



Example 6.3 — Reduced Error Pruning

Full tree (grown on 80% training data):

Training accuracy = 98% (overfitting)

Validation accuracy = 71%

Pruning iteration 1: Consider replacing node N7 (bottom-left) with leaf

Replace with majority class at N7 = 'Yes'

New validation accuracy = 73% → IMPROVED → PRUNE N7 permanently

Pruning iteration 2: Consider replacing N4 (parent of N7) with leaf

Replace with majority class at N4 = 'Yes'

New validation accuracy = 73% → SAME → PRUNE (simpler model preferred)

Pruning iteration 3: Consider replacing N2 (root's right child)

Replace with majority class at N2 = 'No'

New validation accuracy = 69% → DECREASED → KEEP N2

Final pruned tree: smaller, simpler

Training accuracy = 92% (slightly lower)

Test accuracy = 79% (much better than 71%!) → generalises better

Trade-off: We accepted lower training accuracy for better generalisation.

6.5 MDL Principle — Theory

Minimum Description Length — Rigorous Explanation

From information theory: the number of bits to describe X is $-\log_2 P(X)$.

Shorter description = higher probability = simpler/more likely explanation.

MDL Principle: choose hypothesis h that minimises total description length

$$\text{MDL cost} = L(h) + L(D|h)$$

Where:

$L(h)$ = bits to encode the hypothesis (model)
= proxy for MODEL COMPLEXITY
e.g., depth of tree $\times$ bits per node

$L(D|h)$ = bits to encode data GIVEN hypothesis
= proxy for TRAINING ERROR
If h perfectly classifies D : $L(D|h) = 0$
If h makes k errors: $L(D|h) \approx k \times \log_2(n)$ bits

Trade-off:

Simple h : small $L(h)$, large $L(D|h)$ [many errors to encode]
Complex h : large $L(h)$, small $L(D|h)$ [few errors to encode]
MDL finds the optimal balance $\rightarrow$ avoids overfitting

MDL is the algorithmic version of Occam's Razor:

'Among equally good explanations, prefer the simplest one'

Equivalence: MDL $\equiv$ MAP estimation where prior $P(h) \propto 2^{-L(h)}$

Shorter description = stronger prior probability

? EXAM Q: What is the MDL (Minimum Description Length) principle? How is it used in decision trees? What problem does it solve? (6 marks)

MDL Principle: Choose the hypothesis h that minimises the total description length:

$$\text{MDL} = L(h) + L(D|h)$$

Where $L(h)$ = bits to encode the model, $L(D|h)$ = bits to encode errors given model.

Intuition from information theory: the more probable something is, the shorter its optimal code. A good hypothesis makes the data 'easy to describe'.

In Decision Trees:

$L(h)$ = complexity of the tree (number of nodes $\times$ bits per node)

$L(D|h)$ = number of misclassifications $\times \log_2(n)$ bits

A large tree: $L(h)$ is large, $L(D|h)$ is small (few training errors)
A tiny tree: $L(h)$ is small, $L(D|h)$ is large (many training errors)
MDL finds the right tree size that balances both.

Problem it solves: OVERFITTING. MDL provides a principled criterion to stop growing the tree before it memorises training noise, based on information theory rather than arbitrary depth limits.

Formula Quick Reference Sheet

Formula / Concept	Expression
Mitchell's ML Definition	T (task) + P (performance) + E (experience) all improve together
Bayes' Theorem	$P(H D) = P(D H) \cdot P(H) / P(D)$
Shannon Entropy	$H(X) = -\sum p_i \log_2 p_i$ (bits)
Information Gain	$IG(S,A) = H(S) - \sum_v (S_v / S) H(S_v)$
Gain Ratio	$GR(S,A) = IG(S,A) / [-\sum_v (S_v / S) \log_2(S_v / S)]$
Gini Impurity	$Gini(S) = 1 - \sum_k p_k^2$ (range 0 to 0.5 binary)
MDL Principle	$\text{argmin}_h [L(h) + L(D h)]$
Normal Equation	$w^* = (X^T X)^{-1} X^T y$
Ridge Regression	$w^* = (X^T X + \lambda I)^{-1} X^T y$
Gradient Descent Update	$w \leftarrow w - \alpha \cdot (1/n) \cdot X^T (\hat{y} - y)$
Min-Max Normalisation	$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$
Z-score Standardisation	$x' = (x - \mu) / \sigma$
Bias-Variance Decomp	$E[\text{error}] = \text{Bias}^2 + \text{Variance} + \sigma^2_{\text{noise}}$
Sigmoid Function	$\sigma(z) = 1 / (1 + e^{-z})$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
Binary Cross-Entropy Loss	$J = -(1/n) \sum [y \cdot \log \hat{y} + (1-y) \cdot \log(1 - \hat{y})]$
Log-Loss Gradient	$\partial J / \partial w = (1/n) X^T (\hat{y} - y)$ where $\hat{y} = \sigma(Xw)$
Softmax (K-class)	$P(y=k x) = \exp(w_k^T x) / \sum_j \exp(w_j^T x)$
Fisher LDA Solution	$w^* = S W^{-1} (\mu_1 - \mu_2)$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$
F1 Score	$2 \cdot P \cdot R / (P + R)$
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$

Mid-Semester Exam Checklist

✓ Must-Know for 21 June Closed Book Exam

SESSION 1:

- Mitchell's T,P,E definition + 2 examples ready
- 3 types of ML: supervised / unsupervised / reinforcement with examples
- Overfitting vs underfitting: causes and fixes

SESSION 2:

- Bayes theorem: $P(H|D) = P(D|H)P(H)/P(D)$ — compute numerically
- Entropy: $H = -\sum p \cdot \log_2 p$ — calculate for given distribution
- Confusion matrix: TP/FP/FN/TN → compute Accuracy, Precision, Recall, F1

SESSION 3:

- Normal equation: $w = (X^T X)^{-1} X^T y$ — derive and compute
- GD update: $w \leftarrow w - \alpha \cdot (1/n) \cdot X^T (Xw - y)$ — show 1-2 iterations
- Bias² + Variance + Noise — what each means, which is overfitting

SESSION 4:

- MAP decision rule: pick class with max $P(C_k|x)$
- Fisher LDA: $w^* = S W^{-1}(\mu_1 - \mu_2)$
- Generative vs discriminative: key differences

SESSION 5:

- Sigmoid: $\sigma(z) = 1/(1+e^{-z})$, $\sigma'(z) = \sigma(1-\sigma)$, $\sigma(0) = 0.5$
- Log-loss formula and its MLE justification
- Gradient derivation: $\partial J / \partial w = (1/n) X^T (\hat{y} - y)$
- L1 (sparsity) vs L2 (shrinkage) regularisation

SESSION 6:

- Entropy + Information Gain: full calculation (Play Tennis style)
- Gini = $1 - \sum p_k^2$ (different formula from entropy!)
- Reduced error pruning: bottom-up, use validation set
- MDL: $L(h) + L(D|h)$ — what each term means
- Handling continuous attributes: sort → candidate thresholds → compute IG

— End of Study Notes: Sessions 1–6 (v2 with Examples) —

AIMLCZG565 · BITS Pilani WILPD · S2 2025-26